Waleed

Your Phase 1 work and preparation guide

1. Your responsibility

Your area: the ride hierarchy, ride registry, scheduling and ride searches. You connect existing riders and drivers into valid private/shared bookings while preventing time conflicts.

You primarily implement	What you deliver
Ride.java	Abstract common ride fields, immutable ID/time values, getters/setters and pickup-location comparison.
PrivateRide.java	Exactly one rider reference, private-rider access and hasRider.
SharedRide.java	Custom participant list, add/remove checks, emptiness and hasRider.
RideList.java	Every ride registry method; sorted pickup locations; complete name/location searches.
RideSharingSystem.java	Your six methods listed on page 4, especially both conflict-free schedulers.
Main.java	Options 8-12, including private/shared input for Add a ride.

Your reviewer: Nawaf. **Integration tester for your area:** Basam. You review Basam's driver/time/loading code and test Nawaf's lists and rider lifecycle.

First actions:

1. Read IRide, IPrivateRide, ISharedRide, IRideList and your six IRideSharingSystem methods.
2. Agree with Nawaf on the custom list API, rider lookup and history behavior.
3. Agree with Basam on immutable DateTime construction and driver lookup/history behavior.
4. Create the ride hierarchy against these interfaces; finish the registry before connecting full scheduling.

Important participant question: several shared rides use the same pickup location. getSharedRideParticipants accepts only that location, so the selection rule is not fully specified. The provisional plan returns distinct riders across every matching shared ride, preserving first encounter order. Confirm this with the instructor; do not silently select only the first ride.

Use one internal ride-ID generation rule shared by both ride types. CSV files do not contain ride IDs. A pickup location is not a unique ride identifier, and equal compareTo values do not make two different rides duplicates.

This guide follows the existing team split. It is a work checklist, not a claim that your code or tests are already finished. No schedule or effort estimates are included.

2. What you should study

Must know	Where you use it
Abstract classes, inheritance, constructors	Ride is abstract; PrivateRide and SharedRide extend it and implement their supplied interfaces.
Polymorphism and interface types	LinkedList<IRide> stores both ride types; use IRider, IDriver and IDateTime references.
instanceof and safe casts	Inspect private riders or shared participants in name search and cleanup integration.
Comparable and overrides	compareTo(IRide) compares pickup strings. Use the supplied parameter type; do not change it to Ride.
final, references and immutable objects	Ride ID and pickup/drop-off times cannot change. final references must point to immutable DateTime values.
Generics and custom lists	Participants, RideList, search results and per-person ride histories.
int[] and loops	Shared rider IDs, missing-rider checks and duplicate-ID detection.
String equality versus ordering	equals for exact matching; String.compareTo for alphabetical pickup order.
DateTime.compareTo and booleans	Evaluate interval overlap for the assigned driver and every participating rider.

Your minimal linked-list checklist:

☐ Insert rides in pickup-location order at head, middle or end.

☐ Keep multiple rides with equal pickup locations; identify rides using rideId.

☐ Collect all matching rides without returning the same ride twice.

☐ Search/add/remove participant IDs; handle zero/one/many participants safely.

☐ Traverse each person's history without corrupting another traversal's classroom cursor.

☐ Build new result chains with O(1) appends; understand shallow copies.

Overlap means both comparisons are true:

```
pickup1.compareTo(dropoff2) < 0 &&
pickup2.compareTo(dropoff1) < 0
```

If an old ride is 10:00-11:00, a new 10:30-11:30 ride conflicts for the same rider/driver. A new 11:00-12:00 ride is allowed. Use AND and strict <, not OR or <=. Different people may have simultaneous rides.

Useful while building: simple test fixtures, debugger/assertions, compact toString formatting and Git. No entire linked-list chapter is needed: reversal, circular/doubly linked lists, recursion and iterator implementation are not required. Java collection structures are prohibited.

3. Your class implementation checklist

Ride implements IRide: make it abstract. Store final rideId, pickup/drop-off locations, final pickup/drop-off time values and one IDriver reference. Reuse the same registered driver object.

Required common method(s)	Behavior
getRideId	Unique internal identity across private and shared rides.
getPickupLocation / setPickupLocation	Pickup text; changing a stored pickup requires coordinated RideList repositioning.
getPickupTime / getDropoffTime	Return immutable times; there are no time setters.
getDropoffLocation / setDropoffLocation	Drop-off text.
getDriver / setDriver	Driver reference; changing a scheduled assignment requires conflict/history coordination.
compareTo(IRide)	Alphabetical pickup-location comparison only.
hasRider / toString	Rider-specific membership and formatted subtype description; implement/override appropriately.

PrivateRide implements IPrivateRide: keep one IRider; implement getRider, setRider and hasRider by ID. A private ride has exactly one rider. The scheduler validates existence/conflicts before registration.

SharedRide implements ISharedRide:

- Keep a custom LinkedList<IRider>. Implement getParticipants, addParticipant, removeParticipantById and isEmpty, plus hasRider.
- Reject duplicate participant IDs. Return false when an add/remove normally fails.
- Under the agreed design, getParticipants returns a shallow snapshot; structural edits use add/remove methods.
- New shared bookings need at least two distinct riders. After a cascade leaves one rider, keep the ride; only an empty shared ride is deleted.

RideList method	Required behavior
addRide	Enforce unique ride ID and insert in pickup order; allow equal pickup strings.
removeRideById	Remove one unique ride and report true/false.
getAllAlphabetically	Return a pickup-ordered shallow snapshot.
findByPickupLocation	Return every exact matching ride.
findByRiderName	Check every private rider/shared participant using exact full name; add each matching ride only once.
size	Return maintained ride count.

Keep ordinary ride toString output compact, with fixed fields and a participant count if needed. Expanding every participant in every ride makes listing cost depend on total participants, not just the number of rides. Never print inside domain classes or recurse through Person ride histories.

4. Your system methods and booking flow

Implement these **six existing IRideSharingSystem methods** with unchanged signatures.

Method	Your responsibility
schedulePrivateRide	Resolve one rider and driver; check both histories; register one valid private ride.
scheduleSharedRide	Resolve every distinct rider and the driver; check every history; register the complete shared booking.
searchRidesByPickupLocation	Return every ride with the exact pickup string.
searchRidesByRiderName	Search all matching people by full name, including names shared by different riders.
getSharedRideParticipants	Apply the documented, instructor-approved repeated-pickup policy.
getAllRidesAlphabetically	Return all rides in maintained pickup order using a linear traversal.

Successful scheduling has this order:

1. Validate inputs. Recommended defensive rule: pickup precedes drop-off. Reject null/invalid shared-ID arrays and repeated rider IDs.
2. Resolve the driver and all riders from the existing registries. Do not construct replacement Person objects.
3. Check driver history and every rider history using the required overlap test.
4. Only after all ordinary validation passes, construct the ride, fill participants and insert it into RideList.
5. Add that same ride reference once to the driver's history and once to each distinct rider's history. Return true.

On any rejected request, return false without leaving a partial booking or history entry. A shared request with one missing/busy rider fails as a whole. Proposed checks such as rejecting zero-duration intervals should be documented because every invalid-input detail is not spelled out in the interfaces.

Your Main options: 8 -> getAllRidesAlphabetically; 9 -> searchRidesByPickupLocation; 10 -> searchRidesByRiderName; 11 -> getSharedRideParticipants; 12 -> ask private/shared, then call the matching scheduler with locations, times, driver ID and one/multiple rider IDs.

Main constructs the needed DateTime/input values. The system scheduler constructs and registers the Ride. Basam's rides CSV loader calls these same schedulers; it must not bypass validation or insert directly into RideList.

Histories must stay consistent: keep exactly one entry for each accepted ride in the assigned driver's and participating riders' histories. Coordinate whole-ride deletion with Nawaf's removeRideEverywhere helper. Do not independently change getRideHistory to return a snapshot if other members rely on updating its actual list.

5. Tests and Big-O you must deliver

Test	Expected result
Pickup order and ties	AbhaCenter sorts before DhahranPark before RiyadhWest; distinct rides sharing a pickup are both kept.
Unique ride IDs	No collision between private/shared types; no reuse after deletion.
Private booking	One global ride, one rider-history entry and one driver-history entry added.
Shared booking	One global ride; one entry per distinct rider; driver history grows by one only.
Missing rider/driver	false; no changes. Include a missing rider late in a shared array.
Overlap	Reject partial overlap, containment and identical intervals for a shared person.
Adjacency	10:00-11:00 followed by 11:00-12:00 is allowed.
Independent simultaneous rides	Allowed when both riders and drivers are different.
One busy shared rider	Entire booking fails; nobody gains a history entry.
Duplicate/empty shared IDs	Reject under agreed policy without a partial update.
Repeated rider names	Include rides for all riders with that name; return a matching ride only once.
Repeated pickup participants	Handle every matching shared ride according to the confirmed policy; exclude private rides.
Participant removal	Keep one-person shared ride; delete it only when the last participant is removed through the cascade.

Supplied-data checks: 40 rides = 23 private + 17 shared. Searching rides for Karim Al-Zubaidi finds CSV rows 20 and 25. MadinahCentral has three rides, two shared. At DhahranPark the two shared rides contain 7 memberships but 6 distinct riders: 50, 8, 11, 28, 68, 95. Row numbers are not guaranteed ride IDs.

Your analysis	Expected bound under the agreed design
RideList add/remove/copy/pickup search	O(m), assuming O(1) result append.
Rider-name ride search	O(m+P), including shared-participant traversal.
Shared add/remove/hasRider; participant copy	O(q) for q current participants; isEmpty is O(1).
Private scheduling	O(r+d+h_r+h_d+m), simplifying to O(r+d+m).
Shared scheduling	O(d+pr+h_d+sum(h_i)+p^2+m+p); include every rider lookup/history and duplicate checks.
Aggregate participant lookup with linear dedup	O(m+P_L^2), not automatically O(m).

Here r/d/m are riders/drivers/rides; p is new shared participants; P is total stored rider memberships; h is a person's history length; P_L is participant occurrences at matching shared pickups. These bounds assume the agreed implementation. Derive your final loops/helper costs and include constructors/getters/setters/toString in your report section.

6. Team handoffs and your completion checklist

Teammate	What you exchange
Nawaf	Receive the classroom list API, rider existence/search and history behavior. Supply participant operations and RideList removal. Nawaf reviews your scheduling/ride code; you test his list boundaries, rider queries and cascades.
Basam	Receive immutable DateTime and driver registry/history behavior. Supply scheduling methods for his rides loader and handlers for his Main shell. You review his driver/time/loading code; he tests your booking and ride-query integration.

What you should test in Nawaf's code: sorted insertion into empty/singleton lists, collecting repeated names, removing consecutive matches, private-ride cleanup, shared rides with surviving participants and cleanup after removing the final participant.

What you should review in Basam's code: DateTime field order and immutability, plate uniqueness, headerless parsing, enum/date/semicolon conversion, people-before-rides startup and use of your schedulers rather than direct ride insertion.

Group contracts: retain unchanged supplied interfaces; agree constructors and ride-ID generation; use live histories for controlled updates and snapshots for ordinary results; preserve exact full-name matching. Keep a stored ride's ordering/history valid whenever supporting changes to its pickup, driver or riders. No edit menu is required.

Demo practice:

- Explain why compareTo can return 0 for two different rides at the same pickup.
- Trace a shared request through every existence and conflict check.
- Show why exact adjacency is accepted and a failed booking leaves no partial history.
- Explain how both driver deletion and last-participant deletion clean histories.
- Demonstrate a duplicate-name ride search and the confirmed repeated-pickup policy.
- Explain Nawaf's list algorithms and Basam's CSV/DateTime behavior.

You are finished when:

☐ All ride classes and RideList implement their interfaces and inherited methods correctly.

☐ Both schedulers check every involved person before altering stored state.

☐ Your four query methods and Main options 8-12 work.

☐ The repeated-pickup participant rule is confirmed or explicitly documented as provisional.

☐ All your edge-case tests pass and Basam has run end-to-end booking tests.

☐ Nawaf has reviewed your work; you have reviewed Basam's and tested Nawaf's areas.

☐ Your Big-O section includes nested participant/history scans and result-building costs.

☐ Your actual coding, review, testing and integration contributions are recorded.

☐ You can explain the complete program and help verify the source ZIP, PDF report and demo.

Sources: current CSC212_Phase1_Team_Roadmap.pdf, especially Sections 5-8, 12 and 14; supplied RideSharing_Assignment.pdf pp. 1-14; IRide, IPrivateRide, ISharedRide, IRideList, IRideSharingSystem; rides_40.csv. This personal guide preserves the agreed team allocation.